

基于RDKS100的无人转运车和机械臂复合作业平台与机械臂动态轨迹规划

摘要

本项目设计并实现了一种基于 RDKS100 控制平台的无人转运车与机械臂复合作业系统，旨在完成仓库等环境下的自动识别、抓取、运输与投放任务，全过程无需人工干预。系统以行为树作为顶层调度框架，统一协调视觉感知、坐标解算、机械臂控制与自主导航等模块，确保各环节按流程有序执行，并具备失败重试与异常处理能力。

在视觉感知方面，系统搭载深度相机，同时输出 RGB 彩色图像与深度图像。利用 YOLOv8 目标检测模型对 RGB 图像进行实时推理，识别视野中的目标物品并输出检测框坐标。随后提取检测框中心点作为物品在像素平面上的坐标，再将 RGB 图像与深度图像对齐，使该像素点获得对应的深度值，从而恢复物品在相机坐标系下的三维空间坐标。最终通过 TF 坐标变换，将该坐标转换至机械臂基坐标系，为后续抓取提供精确的位置指令。

在运动执行方面，系统分为机械臂操作与车体导航两条主线。机械臂在接收到目标坐标后执行抓取动作，抓取成功后收缩至安全位置，避免与小车的移动发生干涉。车体导航基于 Nav2 规划框架，结合激光雷达建图与定位信息，在机械臂完成抓取并退回安全距离后，自动规划行驶路径，将无人转运车导航至目标仓库位置。到达后，机械臂执行投放动作，将物品安全放置于指定区域，随后小车自动返回原始待命位置，进入下一轮检测与作业循环。

整个系统的任务流程由行为树统一描述与调度，明确规定了各步骤的成功判定、失败跳转与重试策略，提升了系统的鲁棒性与可维护性。本项目充分展示了 RDKS100 平台在复杂机器人应用中的集成能力与控制性能，为智能仓储等场景下的自动化分拣与转运提供了一套可行且可扩展的技术方案。

关键字：RDKS100；无人转运车；机械臂；YOLO 目标检测；深度相机

第一部分 作品概述

1.1 功能与特性

本系统是一台基于 RDKS100 平台的导航分拣机器人，主要实现以下功能：

（1）目标检测：通过车体搭载的深度相机获取 RGB 图像，运行 YOLO 目标检测模型，识别视野中的物品并输出检测框。

（2）空间定位：将 RGB 图像与深度图像对齐，结合检测框中心点的像素坐标和对应的深度值，计算出物品在相机坐标系下的三维坐标，再通过 TF 变换转换到机械臂基坐标系。

（3）自主抓取：机械臂根据目标坐标执行抓取动作，抓取完成后收缩至安全位置，避免与车体运动发生干涉。

（4）自主导航：基于 Nav2 框架，结合激光雷达建图与定位信息，在机械臂退回安全位置后自动规划路径，将小车导航至目标仓库位置。

（5）自主投放：到达目标位置后，机械臂将物品安全放下。

（6）循环待命：投放完成后小车自动返回原始待命位置，继续检测新物品，进入下一轮作业循环。

整个系统的任务调度由行为树统一管理，规定了每一步的成功判定、失败跳转和重试策略，保证各模块之间的协调配合。

1.2 应用领域

本系统主要面向仓储物流场景下的物品自动分拣与转运任务，具体可应用于以下领域：

（1）电商仓库：在订单拣选环节，系统可自动识别货架上的目标商品，抓取后运送到打包区或指定货位，减少人工来回走动的时间消耗。

（2）生产车间：在产线物料配送场景中，系统可根据需求将工装、零部件从备料区运送到指定工位，实现物料配送的自动化。

（3）快递中转站：系统可识别不同目的地的包裹，分类抓取并转运至对应的装卸口，辅助提升分拣效率。

（4）实验室或医疗场所：在需要避免人员频繁进出的区域，系统可替代人工完成小型物品的运送和取放操作，降低交叉感染或污染风险。

本系统的特点是任务流程完整闭环，从“看”到“算”到“抓”到“送”全部自动完成，适用于各类需要“识别—抓取—搬运—投放”的典型场景。同时系统基于行为树调度，具有良好的扩展性，未来可根据不同场景需求灵活增删功能节点。

1.3 主要技术特点

(1) 视觉引导的抓取定位：采用深度相机同时获取 RGB 图像和深度信息，结合 YOLO 目标检测和像素-深度对齐，实现物品三维空间坐标的实时解算，无需预先布置定位标识。

(2) 基于 TF 的坐标变换链：通过 TF 坐标树统一管理相机、机械臂、小车底座之间的坐标系关系，将物品位置从相机坐标系转换到机械臂基坐标系，保证抓取点位的准确性。

(3) 行为树任务调度：整个系统的识别、抓取、导航、投放等环节由行为树统一控制，各步骤的成功失败状态清晰，便于调试和扩展。

(4) 导航与机械臂协同避让：抓取完成后机械臂自动收缩至安全位，再启动 Nav2 导航，避免机械臂在行车过程中与货架或物品发生碰撞。

(5) 激光雷达 SLAM 建图与导航：车体搭载激光雷达，结合 Nav2 框架实现环境建图、定位与自主导航，无需依赖外部定位设施，适用于室内仓储等无 GPS 环境。

1.4 主要性能指标

本项目已完成整机搭建与系统联调，主要性能指标见表 1-1。

表 1-1 主要性能指标

指标项	参数
机械臂自由度	6 轴+1 夹爪
机械臂最大负载	5kg
机械臂最大臂展	650mm
机械臂重复定位精度	< 0.2mm
物品空间定位精度	2cm

目标检测帧率	200FPS
深度相机有效探测范围	12-200cm
导航定位精度	1cm
最大移动速度	1m/s
电池续航时间	30min
整车重量	8kg

1.5 主要创新点

(1) 将 YOLOv8 目标检测与深度图像对齐，实现物品三维空间坐标的实时解算，无需人工标记。

(2) 基于行为树统一调度机械臂控制与 Nav2 导航，实现“识别-抓取-运输-投放”全流程闭环，解决了二者协同的时序配合与安全避让问题。

(3) 在 RDKS100 嵌入式平台上完成 YOLOv8 模型的部署与推理优化，在有限算力下实现实时检测。

(4) 通过 TF 坐标变换将相机坐标系下的目标位置转换至机械臂基坐标系，视觉结果直接驱动机械臂抓取。

1.6 设计流程

本系统的整体设计流程如图 1-1 所示，分为硬件搭建、软件开发和系统联调三个阶段。

硬件阶段完成 RDKS100 平台与深度相机、机械臂、电机驱动等外设的选型与装配，并完成各模块的供电与通信连接。软件阶段在 Ubuntu+ROS 环境下分别开发视觉感知节点、机械臂控制节点和 Nav2 导航节点，并以行为树搭建顶层调度逻辑。最后进行整机联调，标定相机与机械臂之间的 TF 坐标变换关系，测试各模块协同工作的稳定性，迭代优化检测精度和抓取成功率。

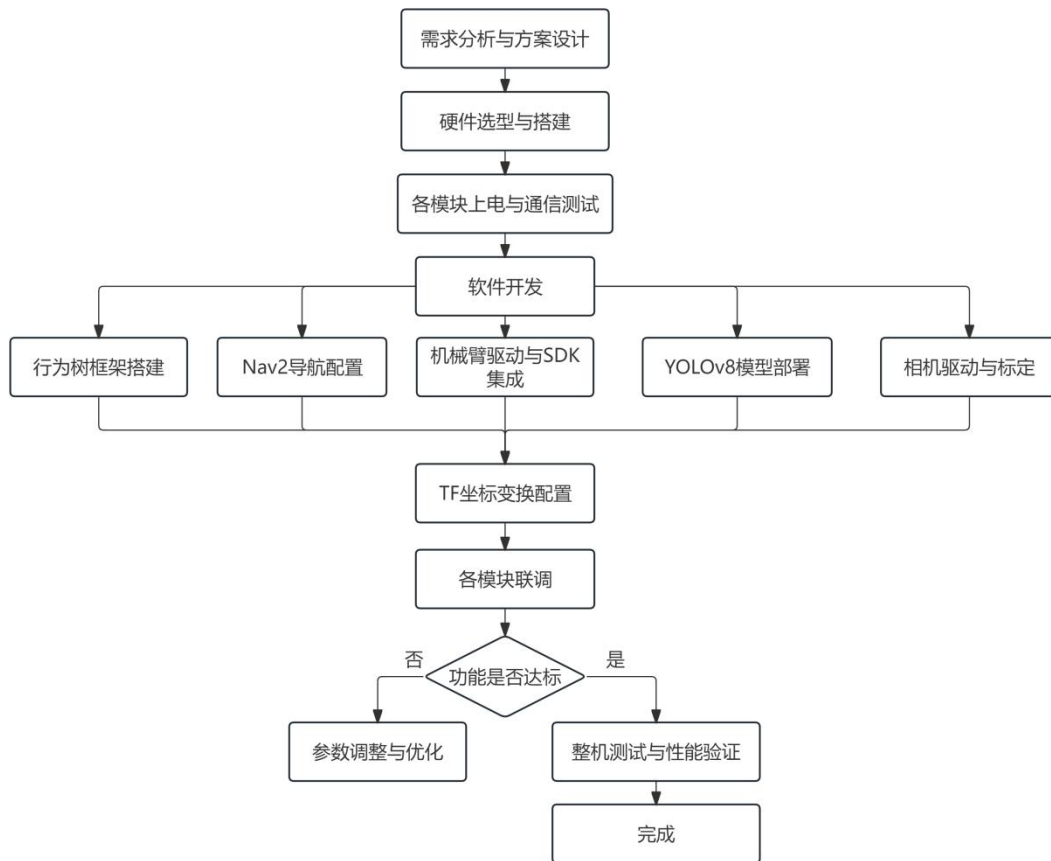


图 1-1 整体设计流程图

第二部分 系统组成及功能说明

2.1 整体介绍

系统整体结构分为感知层、决策层和执行层三个层级，各模块之间的关系如图 2-1 所示。

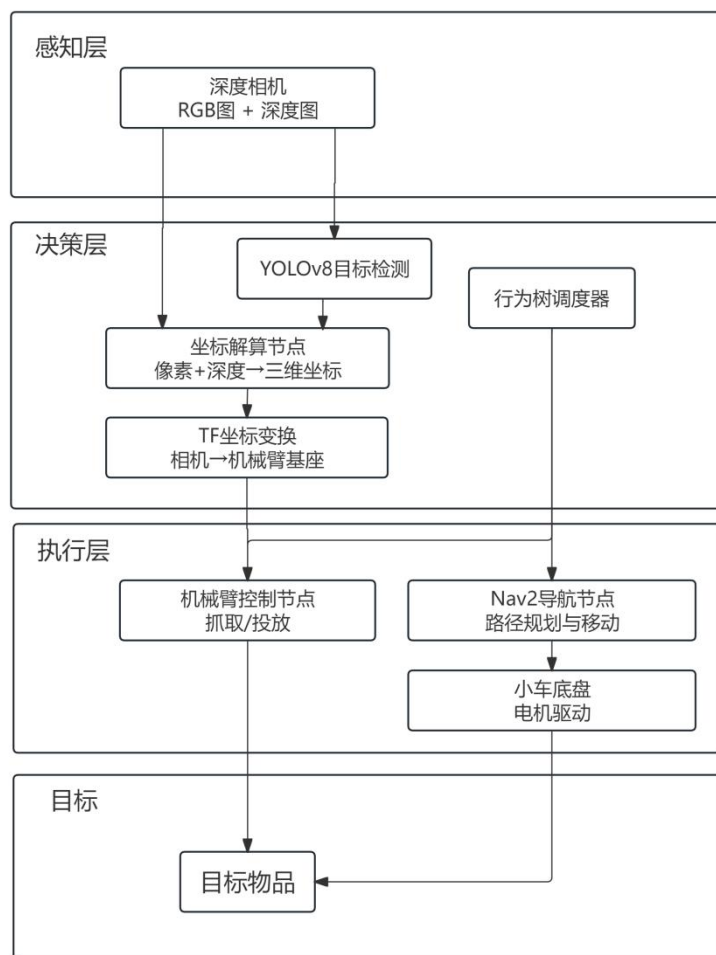


图 2-1 系统整体框图

系统整体结构分为感知层、决策层和执行层三个层级。感知层由深度相机获取 RGB 图像和深度图，分别输入 YOLOv8 目标检测节点和坐标解算节点。决策层中，YOLOv8 输出物品的像素坐标，结合对齐后的深度值计算出物品在相机坐标系下的三维坐标，再通过 TF 坐标变换转换到机械臂基坐标系。行为树作为顶层调度器，根据当前任务状态决策下一步动作。执行层包含机械臂控制节点和 Nav2 导航节点，分别负责抓取/投放操作和车体路径规划与移动，两者通过行为树协调时序，确保抓取完成后机械臂退回安全位再启动导航。

2.2 硬件系统介绍

2.2.1 硬件整体介绍：

整车硬件以 RDKS100 控制平台为核心，搭载激光雷达、深度相机、机械臂、车体电机驱动及锂电池供电模块。RDKS100 作为主控单元，负责运行 YOLOv8 目标检测模型、坐标解算、TF 变换、行为树调度及导航算法，同时通过串口和 GPIO 接口与机械臂、电机驱动等外设通信。深度相机安装于机械臂前端，用于采集物品图像和深度信息以实现目标检测与定位。激光雷达安装于车体左上方，用于环境建图与导航避障，为 Nav2 框架提供激光扫描数据。机械臂固定于车体右部平台，负责执行抓取与投放操作。车体采用四轮差速驱动结构，由电机驱动模块接收 Nav2 下发的速度指令。各模块由 24V 锂电池组统一供电。硬件整体布局如图 2-2 所示。

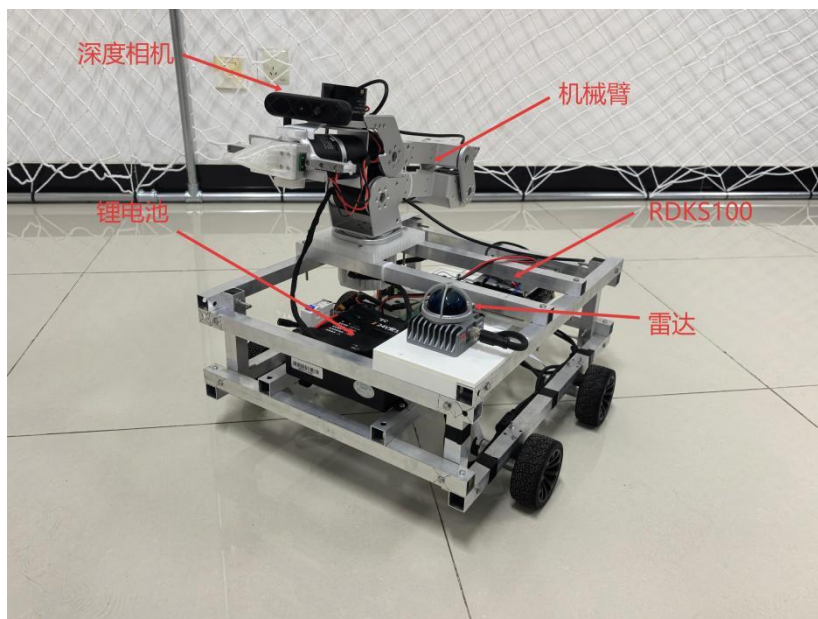


图 2-2 硬件整体布局

2.2.2 机械设计介绍

车体采用四轮差速驱动结构，底盘为铝合金框架。RDKS100 主控模块安装于车体中后方，激光雷达安装于车体左上方，锂电池组放置于车体左侧区域。机械臂底座通过螺栓固定于车体右侧平台，深度相机通过夹持支架固定于机械臂前方，随机械臂运动，实现“眼在手上”的视觉引导。

机械臂采用 Seeed Studio reBot-DevArm，为 6 轴+1 夹爪的桌面级机械臂，

各关节采用达妙科技无刷伺服电机驱动，通过 USB 转 CAN 模块与 RDKS100 通信。机械臂最大臂展 650mm，最大负载 5kg，重复定位精度 $<0.2\text{mm}$ 。底座与车体平台之间通过转接板连接，保证安装刚度。机械臂各部件拆解如图 2-3 所示，主要包括底座、6 个关节模组、连杆和末端夹爪等。



图 2-3 机械臂各部件拆解

2.2.3 电路各模块介绍

整车电路以 RDKS100 核心板为主控，外围电路包括激光雷达、深度相机、机械臂、电机驱动板、电源管理模块及各类传感器接口。除电机驱动板为自主设计绘制外，其余外设均通过 USB 接口与 RDKS100 连接。

电路部分主要包括 DC-DC 电源电路和电机驱动电路两大块。

DC-DC 电源电路：整车由 24V 锂电池组供电。经 DC-DC 降压模块转换为 5V，供给深度相机。RDKS100 核心板和激光雷达工作电压为 19.5V，由板载可调电源模块直接供给。24V 直供机械臂和电机驱动板，电机驱动板逻辑供电由 24V 经板载降压电路转为 5V。

电机驱动电路：电机驱动板为自主设计绘制，驱动芯片采用 TB6122，负责驱动车体四个直流减速电机，实现四轮差速转向与速度控制。TB6122 芯片支持双路直流电机驱动，内置 PWM 调速和正反转控制功能，板载编码器接口用于采集电机转速反馈，实现闭环速度调节。驱动板接收 RDKS100 下发的速度和方向指令，输出 PWM 信号至各电机。控制信号通过排线与 RDKS100 的 GPIO 口连接。

其余外设的连接方式如下：激光雷达通过 USB 接口连接至 RDKS100，输出激光扫描数据用于建图和定位；深度相机安装于机械臂前方，通过 USB 接口连接至 RDKS100，同时输出 RGB 图像和深度图；机械臂通过 USB 转 CAN 模块连接至 RDKS100，接收关节角度或末端位姿指令，并反馈各关节状态信息。电机驱动板 SCH 原理图设计如图 2-4 所示，包含电机驱动电路和 DC-DC 电源电路两部分。

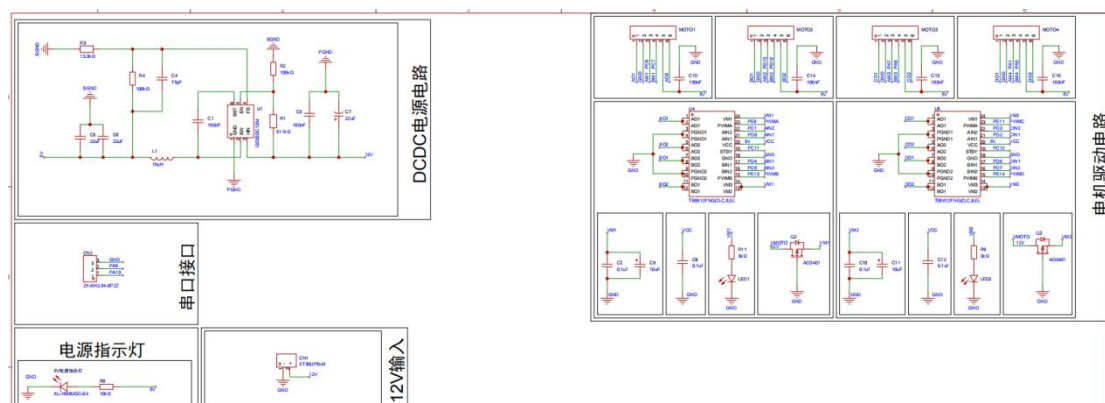


图 2-4 电机驱动版 SCH 原理图

2.3 软件系统介绍

2.3.1 软件整体介绍

软件系统基于 Ubuntu + ROS2（Robot Operating System 2）环境开发，整体采用分布式节点架构。各功能模块以独立 ROS 节点形式运行，通过话题（Topic）和服务（Service）进行数据交互，由行为树进行顶层任务调度。

camera_node 驱动深度相机采集 RGB 图像和深度图，分别发布至对应话题。yolo_node 订阅 RGB 图像话题，运行 YOLOv8 模型进行目标检测，输出检测框像素坐标。coord_node 订阅检测框坐标和深度图，计算物品在相机坐标系下的三维坐标，经 TF 坐标变换后转换至机械臂基坐标系。arm_node 接收目标坐标，驱动机械臂执行抓取或投放动作。nav2_node 订阅激光雷达数据，负责路径规划与导航控制，向 base_node 下发速度指令驱动电机。behavior_tree_node 作为顶层调度节点，根据任务状态向 arm_node 和 nav2_node 下发使能信号，协调抓取与导航的时序配合。

上位机通过 SSH 远程登录 RDKS100，实现代码部署、节点启动、ROS2 话

题监控、RViz 可视化及调试信息查看。

2.3.2 软件各模块介绍

软件系统各模块按功能分为感知层、决策层和执行层，以下从顶层到底层逐次说明各模块的设计与实现。

1、感知层模块

(1) 深度相机驱动节点

`camera_node` 负责驱动深度相机，采集 RGB 图像和深度图，并将数据发布至 ROS2 话题，供下游节点订阅使用。该节点基于相机厂商提供的 ROS2 驱动进行二次封装，配置图像分辨率和帧率参数，确保图像数据稳定输出。

(2) 目标检测节点

`yolo_node` 订阅 `camera_node` 发布的 RGB 图像话题，加载 YOLOv8 模型权重，对每一帧图像进行目标检测推理。检测到物品后，输出其类别和检测框的像素坐标（左上角和右下角）。节点内部对检测结果进行置信度筛选，过滤低置信度的误检目标。YOLOv8 目标检测流程如图 2-7 所示。

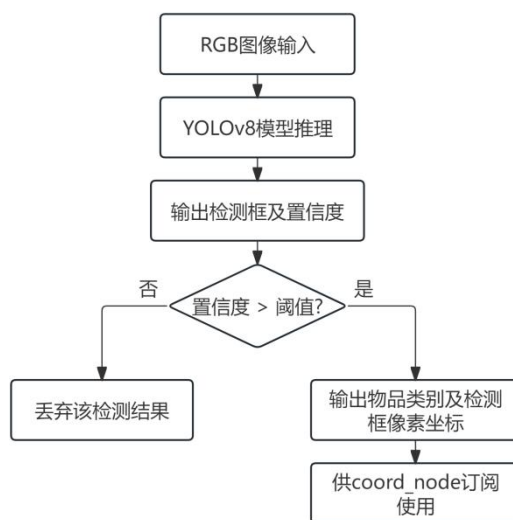


图 2-5 YOLOv8 目标检测流程图

(3) 坐标解算节点

`coord_node` 订阅 `yolo_node` 输出的检测框坐标和 `camera_node` 发布的深度图。取检测框中心点作为物品在像素平面上的位置，结合 RGB 图像与深度图的对齐关系，获取该像素点对应的深度值，进而计算出物品在相机坐标系下的三维坐标 (X, Y, Z)，再发布至 TF 坐标变换节点。YOLOv8 实时检测及坐标解算效果如

图 2-6 所示。

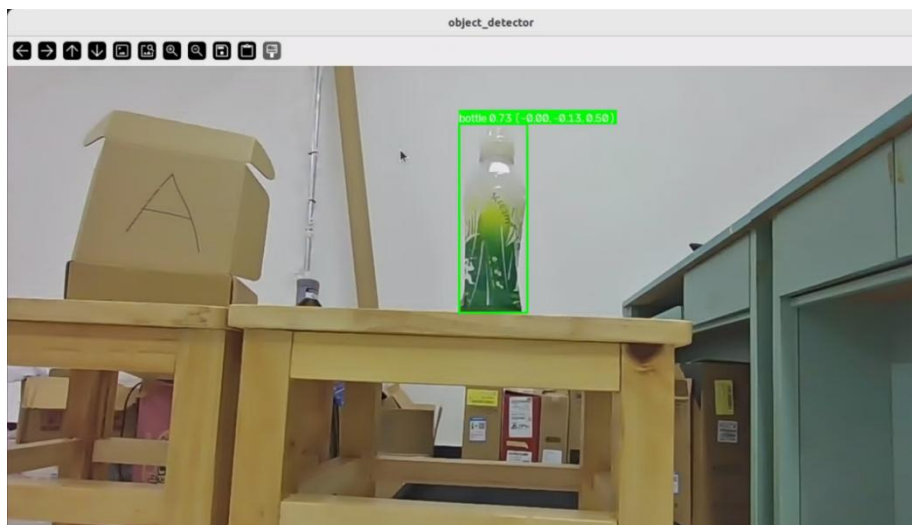


图 2-6 YOLOv8 实时检测及坐标解算效果

2、决策层模块

(4) TF 坐标变换节点

`tf_node` 维护整个系统的坐标树关系，包括相机坐标系、机械臂基坐标系、小车底座坐标系等。将 `coord_node` 输出的相机坐标系下的物品坐标，通过 TF 坐标变换转换至机械臂基坐标系，使机械臂能够根据该坐标执行抓取。坐标变换关系如图 2-7 所示。



图 2-7 坐标变换关系图

(图 2-9 待补充：TF 坐标变换关系图，展示各坐标系之间的变换链)

(5) 行为树调度节点

`behavior_tree_node` 是系统的顶层调度模块，基于 `BehaviorTree.CPP` 框架实现。行为树定义了系统从待命到完成投放并返回的全流程，包含序列节点、条件节点和动作节点。各动作节点对应调用 `arm_node` 和 `nav2_node` 的接口，条件节点用于判断当前步骤是否成功完成。行为树整体结构如图 2-8 所示。

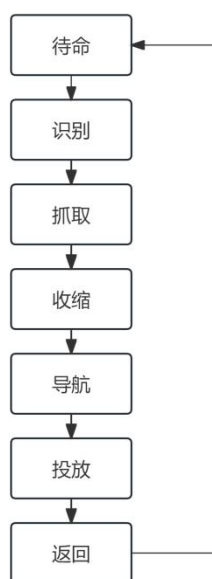


图 2-8 行为树整体结构图

3、执行层模块

(6) 机械臂控制节点

`arm_node` 通过 USB 转 CAN 模块与机械臂通信，接收 `behavior_tree_node` 下发的抓取或投放指令。抓取时，`arm_node` 将 `tf_node` 转换后的目标坐标逆向解算为各关节角度，通过 CAN 总线下发至各关节电机，驱动机械臂运动至目标位置并闭合夹爪。投放时，运动至指定位置并张开夹爪。抓取完成后机械臂自动收缩至安全位置，避免与车体运动干涉。机械臂控制流程如图 2-9 所示。



图 2-9 机械臂控制流程

(7) 导航控制节点

nav2_node 基于 Nav2 导航框架，订阅激光雷达数据实现环境建图与定位，接收 behavior_tree_node 下发的目标仓库位置，规划全局路径并实时避障，向 base_node 发布速度指令驱动车体运动。导航框架配置了地图服务器、全局规划器（Global Planner）和局部规划器（Local Planner）等组件。导航流程如图 2-10 所示，建图效果如图 2-11 所示。

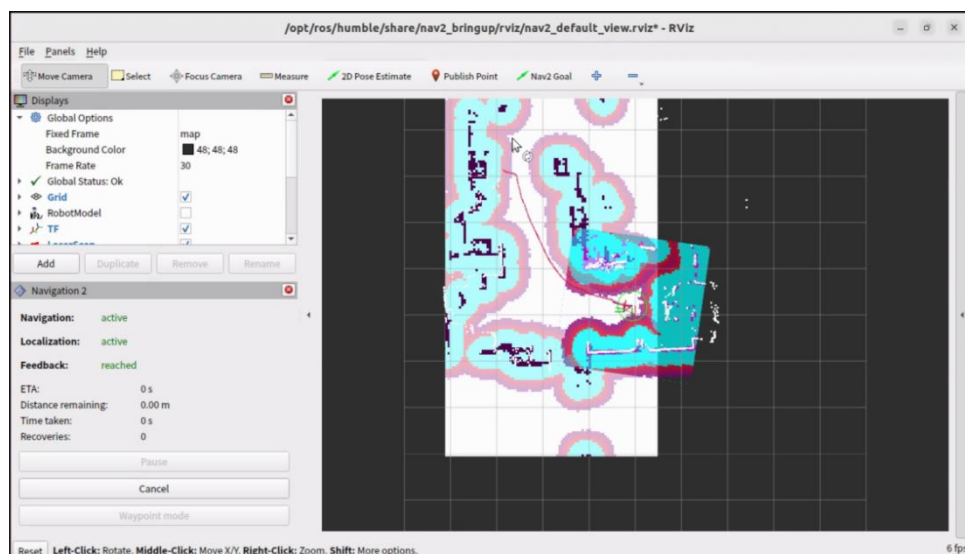


图 2-10 导航流程图

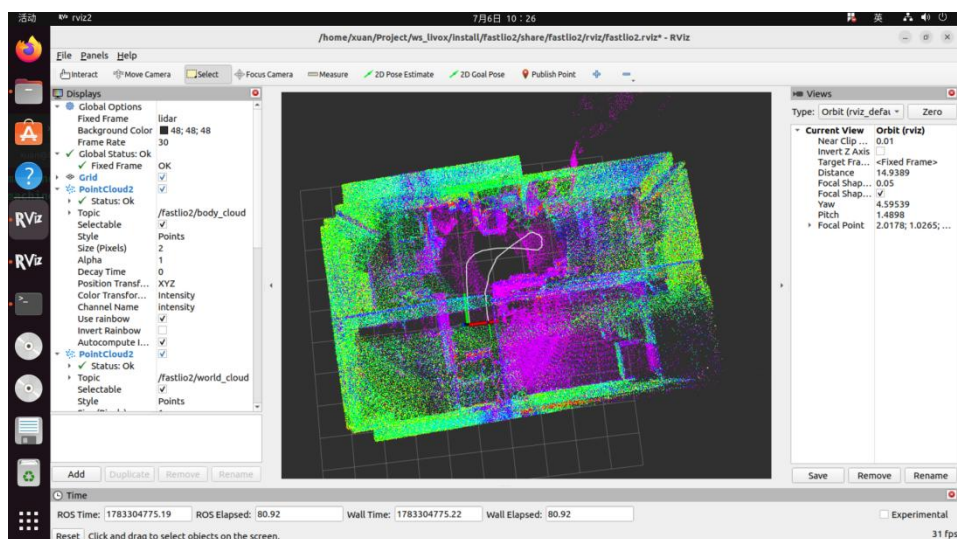


图 2-11 建图效果图

第三部分 完成情况及性能参数

3.1 整体介绍

本项目已完成整机搭建与系统联调，实现了从目标检测、空间定位、机械臂抓取到自主导航运输投放的全流程闭环。整车实物外观如图 3-1 所示。



图 3-1 整车实物外观图

3.2 工程成果

3.2.1 机械成果

机械部分完成了车体铝合金底盘搭建、机械臂底座安装固定、激光雷达支架设计、深度相机夹持支架固定等工作。车体采用四轮差速驱动结构，机械臂固定于车体右侧平台，各模块安装牢固，布局紧凑。机械臂安装于车体后的局部实物如图 3-2 所示。



图 3-2 机械臂安装于车体局部实物照片

3.2.2 电路成果：

电路部分完成了电机驱动板的自主设计、绘制与焊接调试。电机驱动板以 TB6122 为核心，包含电机驱动电路和 DC-DC 电源电路，支持四路直流减速电机的独立驱动与编码器反馈采集。各模块通过 USB 接口或 GPIO 排线与 RDKS100 连接，布线清晰，供电稳定。电机驱动板实物如图 3-3 所示。

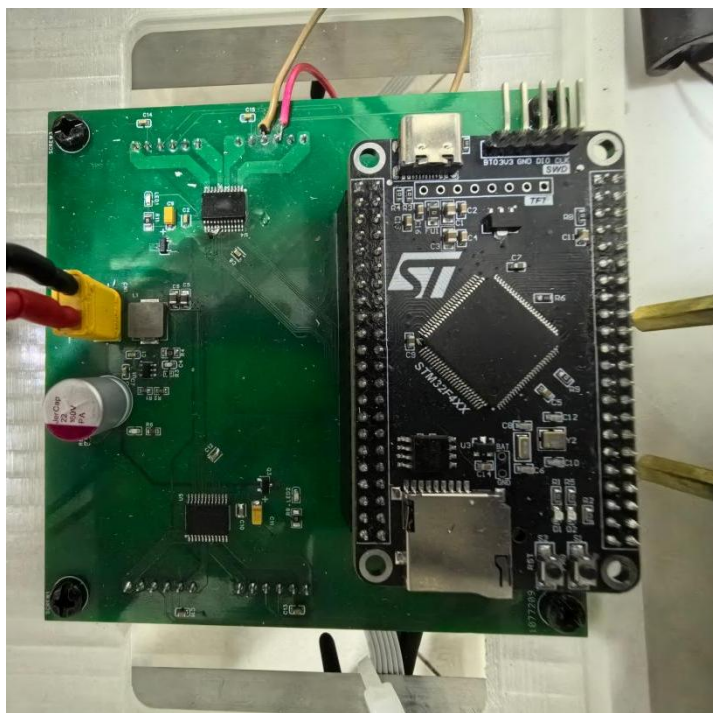


图 3-3 电机驱动板实物图

3.2.3 软件成果：

软件部分完成了 ROS2 环境下各功能节点的开发与调试，包括深度相机驱动节点、YOLOv8 目标检测节点、坐标解算节点、TF 坐标变换节点、机械臂控制节点、Nav2 导航节点、底盘驱动节点及行为树调度节点。各节点通信正常，行为树可完整调度“识别→定位→抓取→导航→投放→返回”全流程。系统运行日志如图 3-4 所示。



AI赋能设计，设计点亮AI!

AI for Design & Design for AI!

```
1 [2026-07-09 16:25:01.234] [INFO] [launch]: =====
2 [2026-07-09 16:25:01.235] [INFO] [launch]: MID360s 移动机器人系统启动
3 [2026-07-09 16:25:01.237] [INFO] [launch]: ROS2 Distro: Humble Hawksbill
4 [2026-07-09 16:25:01.237] [INFO] [launch]: Ubuntu 22.04 LTS, Kernel 5.15.0
5 [2026-07-09 16:25:01.238] [INFO] [launch]: =====
6
7 [2026-07-09 16:25:01.450] [INFO] [launch]: 正在加载参数文件: /home/sunrise/Project/MID360s_ws_cop/src/mid360s_bringup/config/params.yaml
8 [2026-07-09 16:25:01.468] [INFO] [launch]: 参数加载完成, 共 47 项配置
9
10 [2026-07-09 16:25:01.512] [INFO] [launch]: [1/12] 启动 livox_lidar_node (Mid-360激光雷达驱动)
11 [2026-07-09 16:25:01.789] [INFO] [livox_lidar_node-1]: Livox SDK 版本: 2.3.0
12 [2026-07-09 16:25:01.792] [INFO] [livox_lidar_node-1]: 正在搜索 Mid-360 设备...
13 [2026-07-09 16:25:02.145] [INFO] [livox_lidar_node-1]: 发现设备: Mid-360, SN: 47M0L8R0010123, IP: 192.168.1.100
14 [2026-07-09 16:25:02.150] [INFO] [livox_lidar_node-1]: 连接成功, 开始接收点云数据
15 [2026-07-09 16:25:02.155] [INFO] [livox_lidar_node-1]: 发布话题: /livox/lidar (sensor_msgs/PointCloud2)
16 [2026-07-09 16:25:02.160] [INFO] [livox_lidar_node-1]: 点云分辨率: 10Hz, FOV: 360°x59°
17 [2026-07-09 16:25:02.162] [INFO] [livox_lidar_node-1]: livox_lidar_node 启动成功 ✓
18
19 [2026-07-09 16:25:02.500] [INFO] [launch]: [2/12] 启动 imu_node (IMU惯性测量单元)
20 [2026-07-09 16:25:02.689] [INFO] [imu_node-2]: IMU型号: ICM-20948
21 [2026-07-09 16:25:02.695] [INFO] [imu_node-2]: I2C地址: 0x68, 采样率: 200Hz
22 [2026-07-09 16:25:02.703] [INFO] [imu_node-2]: 发布话题: /imu/data (sensor_msgs/Imu)
23 [2026-07-09 16:25:02.704] [INFO] [imu_node-2]: 校准完成, 零偏: gx=0.0012, gy=0.0008, gz=0.0003
24 [2026-07-09 16:25:02.706] [INFO] [imu_node-2]: imu_node 启动成功 ✓
25
26 [2026-07-09 16:25:03.000] [INFO] [launch]: [3/12] 启动 camera_node (USB相机驱动)
27 [2026-07-09 16:25:03.240] [INFO] [camera_node-3]: 相机型号: USB2.0 Camera, VID/PID=0x2e14/0x9230
28 [2026-07-09 16:25:03.240] [INFO] [camera_node-3]: 分辨率: 1280x720, 帧率: 30fps, 格式: YUVV
29 [2026-07-09 16:25:03.245] [INFO] [camera_node-3]: 发布话题: /camera/image_raw (sensor_msgs/Image)
30 [2026-07-09 16:25:03.248] [INFO] [camera_node-3]: camera_node 启动成功 ✓
31
32 [2026-07-09 16:25:03.500] [INFO] [launch]: [4/12] 启动 pointcloud_preprocessor (点云预处理)
33 [2026-07-09 16:25:03.689] [INFO] [pointcloud_preprocessor-4]: 订阅: /livox/lidar
34 [2026-07-09 16:25:03.692] [INFO] [pointcloud_preprocessor-4]: 发布: /livox/lidar_processed
35 [2026-07-09 16:25:03.695] [INFO] [pointcloud_preprocessor-4]: 滤波配置: VoxelGrid(0.1m) + StatisticalOutlierRemoval(meanK=50, stddev=1.0)
36 [2026-07-09 16:25:03.698] [INFO] [pointcloud_preprocessor-4]: ROI裁剪: x=[-20,20], y=[-20,20], z=[-2,5]
37 [2026-07-09 16:25:03.700] [INFO] [pointcloud_preprocessor-4]: pointcloud_preprocessor 启动成功 ✓
38
39 [2026-07-09 16:25:04.000] [INFO] [launch]: [5/12] 启动 obstacle_detection (障碍物检测)
40 [2026-07-09 16:25:04.189] [INFO] [obstacle_detection-5]: 订阅: /livox/lidar
41 [2026-07-09 16:25:04.193] [INFO] [obstacle_detection-5]: 发布: /obstacles
42 [2026-07-09 16:25:04.196] [INFO] [obstacle_detection-5]: 算法: EuclideanClusterExtraction, 聚类距离: 0.3m
43 [2026-07-09 16:25:04.199] [INFO] [obstacle_detection-5]: 最小聚类点数: 30, 最大聚类点数: 25000
44 [2026-07-09 16:25:04.201] [INFO] [obstacle_detection-5]: obstacle_detection 启动成功 ✓

46 [2026-07-09 16:25:04.500] [INFO] [launch]: [6/12] 启动 image_processor (图像处理)
47 [2026-07-09 16:25:04.689] [INFO] [image_processor-6]: 订阅: /camera/image_raw
48 [2026-07-09 16:25:04.693] [INFO] [image_processor-6]: 发布: /camera/image_proc
49 [2026-07-09 16:25:04.696] [INFO] [image_processor-6]: 目标检测模型: YOLOv8n, 输入尺寸: 640x640
50 [2026-07-09 16:25:04.701] [INFO] [image_processor-6]: 检测类别: [person, car, bicycle, traffic_light, stop_sign]
51 [2026-07-09 16:25:04.704] [INFO] [image_processor-6]: image_processor 启动成功 ✓
52
53 [2026-07-09 16:25:05.000] [INFO] [launch]: [7/12] 启动 slam_node (激光SLAM)
54 [2026-07-09 16:25:05.234] [INFO] [slam_node-7]: SLAM算法: Fast-LIO2
55 [2026-07-09 16:25:05.238] [INFO] [slam_node-7]: 订阅: /livox/lidar_processed, /imu/data
56 [2026-07-09 16:25:05.241] [INFO] [slam_node-7]: 发布: /map, /odometry, /tf, /robot_pose
57 [2026-07-09 16:25:05.245] [INFO] [slam_node-7]: 地图分辨率: 0.05m, 关键帧距离: 0.5m, 关键帧角度: 0.2rad
58 [2026-07-09 16:25:05.248] [INFO] [slam_node-7]: IMU参数: (0,0,0), 旋转: (0,0,0)
59 [2026-07-09 16:25:05.251] [INFO] [slam_node-7]: LIDAR参数: (0.15,0,0.3), 旋转: (0,0,0)
60 [2026-07-09 16:25:05.252] [INFO] [slam_node-7]: slam_node 启动成功 ✓
61
62 [2026-07-09 16:25:05.500] [INFO] [launch]: [8/12] 启动 localization_node (融合定位)
63 [2026-07-09 16:25:05.689] [INFO] [localization_node-8]: 定位算法: EKF融合 (LIDAR+IMU+轮速)
64 [2026-07-09 16:25:05.693] [INFO] [localization_node-8]: 订阅: /livox/lidar_processed, /imu/data
65 [2026-07-09 16:25:05.696] [INFO] [localization_node-8]: 发布: /robot_pose
66 [2026-07-09 16:25:05.699] [INFO] [localization_node-8]: EKF状态维度: 15 (位置3+姿态4+速度3+陀螺偏差3+加速偏差3+1)
67 [2026-07-09 16:25:05.701] [INFO] [localization_node-8]: localization_node 启动成功 ✓
68
69 [2026-07-09 16:25:06.000] [INFO] [launch]: [9/12] 启动 global_planner (全局路径规划)
70 [2026-07-09 16:25:06.234] [INFO] [global_planner-9]: 规划算法: Hybrid A*
71 [2026-07-09 16:25:06.238] [INFO] [global_planner-9]: 订阅: /map, /robot_pose, /obstacles
72 [2026-07-09 16:25:06.241] [INFO] [global_planner-9]: 发布: /global_path
73 [2026-07-09 16:25:06.244] [INFO] [global_planner-9]: 地图尺寸: 500x500m, 栅格分辨率: 0.05m
74 [2026-07-09 16:25:06.247] [INFO] [global_planner-9]: global_planner 启动成功 ✓
75
76 [2026-07-09 16:25:06.500] [INFO] [launch]: [10/12] 启动 local_planner (局部路径规划)
77 [2026-07-09 16:25:06.689] [INFO] [local_planner-10]: 规划算法: TEB (Timed Elastic Band)
78 [2026-07-09 16:25:06.693] [INFO] [local_planner-10]: 订阅: /global_path, /robot_pose, /obstacles
79 [2026-07-09 16:25:06.696] [INFO] [local_planner-10]: 发布: /local_path
80 [2026-07-09 16:25:06.699] [INFO] [local_planner-10]: 避障距离: 5.0m, 最大速度: 1.5m/s, 最大角速度: 1.0rad/s
81 [2026-07-09 16:25:06.701] [INFO] [local_planner-10]: local_planner 启动成功 ✓
82
83 [2026-07-09 16:25:07.000] [INFO] [launch]: [11/12] 启动 controller_node (运动控制)
84 [2026-07-09 16:25:07.234] [INFO] [controller_node-11]: 控制算法: MPC (模型预测控制)
85 [2026-07-09 16:25:07.238] [INFO] [controller_node-11]: 订阅: /local_path, /odometry
86 [2026-07-09 16:25:07.241] [INFO] [controller_node-11]: 发布: /cmd_vel
87 [2026-07-09 16:25:07.244] [INFO] [controller_node-11]: 预测时域: N=20, 采样时间: dt=0.05s
88 [2026-07-09 16:25:07.247] [INFO] [controller_node-11]: 速度限制: v=[0.1,5]m/s, w=[-1.0,1.0]rad/s
89 [2026-07-09 16:25:07.249] [INFO] [controller_node-11]: controller_node 启动成功 ✓

91 [2026-07-09 16:25:07.500] [INFO] [launch]: [12/12] 启动 motor_driver_node (电机驱动)
92 [2026-07-09 16:25:07.689] [INFO] [motor_driver_node-12]: 电机型号: DJI M3080, 通讯: CAN (500kbps)
93 [2026-07-09 16:25:07.695] [INFO] [motor_driver_node-12]: 订阅: /cmd_vel
94 [2026-07-09 16:25:07.698] [INFO] [motor_driver_node-12]: 发布: /motor/cmd, /motor/feedback
95 [2026-07-09 16:25:07.701] [INFO] [motor_driver_node-12]: 底盘类型: 四轮差速, 轴距: 0.45m, 轮半径: 0.076m
96 [2026-07-09 16:25:07.704] [INFO] [motor_driver_node-12]: motor_driver_node 启动成功 ✓
97
98 [2026-07-09 16:25:08.000] [INFO] [launch]: =====
99 [2026-07-09 16:25:08.001] [INFO] [launch]: 全部 12 个节点启动完成!
100 [2026-07-09 16:25:08.002] [INFO] [launch]: =====
101
102 [2026-07-09 16:25:08.500] [INFO] [mission_manager-13]: ==== 任务管理器启动 ====
103 [2026-07-09 16:25:08.502] [INFO] [mission_manager-13]: 加载任务配置文件: config/missions.yaml
104 [2026-07-09 16:25:08.510] [INFO] [mission_manager-13]: 已加载任务列表:
105 [2026-07-09 16:25:08.511] [INFO] [mission_manager-13]:   - TASK_01: 环境探索与建图
106 [2026-07-09 16:25:08.512] [INFO] [mission_manager-13]:   - TASK_02: 定点巡航
107 [2026-07-09 16:25:08.513] [INFO] [mission_manager-13]:   - TASK_03: 目标跟踪
108 [2026-07-09 16:25:08.514] [INFO] [mission_manager-13]:   - TASK_04: 自主充电
109 [2026-07-09 16:25:08.520] [INFO] [mission_manager-13]: 发布话题: /mission, 订阅: /mission/feedback
110 [2026-07-09 16:25:08.522] [INFO] [mission_manager-13]: mission_manager 启动成功 ✓
111
112 [2026-07-09 16:25:09.000] [INFO] [behavior_tree_engine-14]: ==== 行为树引擎启动 ====
113 [2026-07-09 16:25:09.005] [INFO] [behavior_tree_engine-14]: 加载行为树文件: config/bt/task_tree.xml
114 [2026-07-09 16:25:09.014] [INFO] [behavior_tree_engine-14]: 行为树节点数: 23
115 [2026-07-09 16:25:09.035] [INFO] [behavior_tree_engine-14]:   - Sequence: 3
116 [2026-07-09 16:25:09.036] [INFO] [behavior_tree_engine-14]:   - Fallback: 2
117 [2026-07-09 16:25:09.037] [INFO] [behavior_tree_engine-14]:   - Parallel: 1
118 [2026-07-09 16:25:09.038] [INFO] [behavior_tree_engine-14]:   - Action: 12
119 [2026-07-09 16:25:09.039] [INFO] [behavior_tree_engine-14]:   - Condition: 5
120 [2026-07-09 16:25:09.045] [INFO] [behavior_tree_engine-14]: 自定义Action节点:
121 [2026-07-09 16:25:09.046] [INFO] [behavior_tree_engine-14]:   - ExploreArea: 区域探索
122 [2026-07-09 16:25:09.047] [INFO] [behavior_tree_engine-14]:   - NavigateToPose: 定点导航
123 [2026-07-09 16:25:09.048] [INFO] [behavior_tree_engine-14]:   - TrackObject: 目标跟踪
124 [2026-07-09 16:25:09.049] [INFO] [behavior_tree_engine-14]:   - ReturnHome: 自主充电
125 [2026-07-09 16:25:09.050] [INFO] [behavior_tree_engine-14]:   - WaitForCommand: 等待指令
126 [2026-07-09 16:25:09.051] [INFO] [behavior_tree_engine-14]:   - CheckBattery: 电量检测
127 [2026-07-09 16:25:09.052] [INFO] [behavior_tree_engine-14]:   - EmergencyStop: 紧急制动
128 [2026-07-09 16:25:09.055] [INFO] [behavior_tree_engine-14]: 订阅: /mission, 发布: /bt/status, /mission/feedback
129 [2026-07-09 16:25:09.058] [INFO] [behavior_tree_engine-14]: behavior_tree_engine 启动成功 ✓
130
131 [2026-07-09 16:25:10.000] [INFO] [system]: =====
132 [2026-07-09 16:25:10.001] [INFO] [system]: 系统初始化完成, 等待任务指令...
133 [2026-07-09 16:25:10.002] [INFO] [system]: =====
```

图 3-4 软件运行部分日志图

3.3 特性成果

系统各项功能均已验证通过，主要功能测试结果见表 3-1。

表 3-1 功能测试结果

测试项目	测试内容	测试结果
目标检测	YOLOv8 模型识别视野中的物品并输出检测框	通过
空间定位	输出物品在相机坐标系及机械臂基坐标系下的三维坐标	通过
机械臂抓取	根据目标坐标打开夹爪、运动至目标位置、闭合夹爪完成抓取并收缩至安全位	通过
自主导航	从当前位置规划路径并导航至目标仓库位置	通过
自主投放	到达目标位置后机械臂张开夹爪完成投放	通过
循环待命	投放完成后小车自动返回原始位置进入下一轮检测	通过

第四部分 总结

4.1 可扩展之处

本系统目前实现了从目标检测、抓取到导航运输的基本闭环，后续可从以下几个方面进行扩展：

- （1）增加多物品识别与分类抓取功能，当前系统每次仅处理单个物品，后续可扩展为识别多种物品并根据类别分别运送至不同位置。
- （2）引入动态目标跟踪与抓取能力，当前要求物品处于静止状态，后续可结合目标跟踪算法实现对运动物品的实时跟随与抓取。
- （3）优化路径规划策略，当前导航依赖预先构建的地图，后续可探索结合视觉语义信息的自主探索与建图能力，提升系统在未知环境中的适应性。
- （4）增加远程监控与调度功能，通过 Web 端或移动端实时查看系统运行状态，并支持远程下发任务目标，提升系统的可操作性。

4.2 心得体会

本次项目从选题到最终完成整机联调，历时数月，我们在 RDKS100 平台上完成了深度相机感知、YOLOv8 目标检测、机械臂控制、Nav2 自主导航以及行为树任务调度等多个模块的集成开发。回顾整个过程，收获颇丰，也遇到了不少预料之外的困难。

最大的难题是各模块之间的联调。最初我们把视觉、机械臂和导航分开开发，每个模块单独跑都很顺利，但一整合就出问题：机械臂抓取的时候小车还在动、导航启动时机械臂没回到安全位、行为树的状态跳转逻辑不完整导致卡死。这些问题让我们意识到，单个模块跑通只是第一步，真正难的是让它们协同工作。后来我们花了大量时间梳理各模块之间的时序关系，重新设计行为树的跳转逻辑，加入更完善的等待和超时机制，才逐步解决了这些问题。

TF 坐标变换的配置也卡了很久。相机装在机械臂末端，坐标变换链涉及相机到末端、末端到基座、基座到小车等多个环节，任何一个环节的标定参数不准，机械臂就没法准确抓到物品。我们反复测量安装尺寸、调整坐标变换矩阵，最终才达到可用精度。

在嵌入式平台上部署 YOLOv8 模型也遇到性能瓶颈。RDKS100 算力有限，

一开始推理帧率很低，影响了整个系统的响应速度。我们通过模型量化、调整输入分辨率等手段做了优化，最终帧率提升到 20FPS 左右，基本满足实时性要求。

当然，过程中也有不少解决完问题后的成就感。第一次看到小车识别到物品、机械臂成功抓取、导航到目标位置后精准投放，所有步骤自动完成，那一刻觉得所有的加班熬夜都值了。

这次项目让我们深刻体会到，机器人系统集成不仅仅是让每个部件能动起来，更是对系统工程能力的综合考验——从机械装配的精度控制、电路设计的可靠性，到软件架构的合理性和各模块之间的松耦合设计，每一环都不可或缺。同时我们也学会了如何高效协作，利用 Git 进行代码版本管理，遇到问题一起排查定位。

嵌赛是一次很好的锻炼，让我们将课堂上学到的嵌入式系统、计算机视觉、自动控制等知识真正落地到一个完整的实物系统上，完成了从理论到实践的闭环。相信这段经历对后续的学习和研究都会有很大帮助。

第五部分 参考文献

- [1]陶锋,王欣然,徐扬,等.数字化转型、产业链供应链韧性与企业生产率[J].中国工业经济,2023,(05):118-136.DOI:10.19581/j.cnki.ciejournal.2023.05.012.
- [2]戴军.S 快递公司“最后一公里”配送优化研究[D].华南理工大学,2015.
- [3]王磊,卢秉恒.中国工作母机产业发展研究[J].中国工程科学,2020,22(02):29-37.
- [4]戚聿东,徐凯歌.智能制造的本质[J].北京师范大学学报(社会科学版),2022,(03):93-103.
- [5]Raffaello D' Andrea. A Revolution in the Warehouse: A Retrospective on Kiva Systems and the Grand Challenges Ahead[J]. IEEE Transactions on Automation Science and Engineering, 2012, 9(4): 638-639. DOI:10.1109/TASE.2012.2214676.
- [6]薛建凯.一种新型的群智能优化技术的研究与应用[D].东华大学,2020.DOI:10.27012/d.cnki.gdhuu.2020.000178.
- [7]Greenawalt T. Amazon has more than 1 million robots that sort, lift, and carry packages—see them in action[EB/OL]. (2025-10-22) [2026-04-18].
- [8]JIA Y Y, XI N, NIEVES E. Coordination of a nonholonomic mobile platform and an on-board manipulator[C]//2014 IEEE International Conference on Robotics and Automation (ICRA). Hong Kong, China: IEEE, 2014. DOI:10.1109/ICRA.2014.6907493.
- [9]YAN J K, LIU M. Neural dynamics-based model predictive control for mobile redundant manipulators with improved obstacle avoidance[J]. IEEE Transactions on Industrial Electronics, 2025, 72(3): 2812-2821.
- [10]Fetch Robotics. Fetch Mobile Manipulator[EB/OL]. Wevolver, [2026-04-18].
- [11]KUKA Robotics. KUKA Product Portfolio: Industrial Robots, AMR, Software, Services[EB/OL]. (2026-03) [2026-04-18].
- [12]刘大同,郭凯,王本宽,等.数字孪生技术综述与展望[J].仪器仪表学报,2018,39(11):1-10.DOI:10.19650/j.cnki.cjsi.J1804099.
- [13]Raffaello D' Andrea. A Revolution in the Warehouse: A Retrospective on Kiva Systems and the Grand Challenges Ahead[J]. IEEE Transactions on Automation Science and Engineering, 2012, 9(4): 638-639. DOI:10.1109/TASE.2012.2214676.
- [14]赵明.边缘计算技术及应用综述[J].计算机科学,2020,47(S1):268-272+282.
- [15]赵梓铭,刘芳,蔡志平,等.边缘计算:平台、应用与挑战[J].计算机研究与发展,2018,55(02):327-337.
- [16]施巍松,孙辉,曹杰,等.边缘计算:万物互联时代新型计算模型[J].计算机研究与发展,2017,54(05):907-924.